

CSL7640 : ASSIGNMENT-IV: Sentiment Analysis

B22CS001, B22CS093, B22BB016

13th April 2025



1 Introduction

Sentiment Analysis is a pivotal task in **Natural Language Processing (NLP)** that focuses on identifying and categorizing sentiments expressed in textual data, such as movie reviews or social media posts. In this assignment, we aim to build robust sentiment classification systems capable of handling both **binary** (positive/negative) and **multi-class** (positive/negative/neutral) sentiment analysis tasks. To accomplish this, we explore two primary deep learning architectures:

- **Feed-Forward Neural Network (FFNN)** – A traditional neural architecture that processes fixed-length, one-hot encoded representations of text and learns hierarchical feature representations through multiple hidden layers.
- **Recurrent Neural Network with LSTM (Long Short-Term Memory)** – A sequential architecture that captures temporal dependencies across words in a sentence. We utilize the final hidden state of the LSTM as a summary vector for sentiment classification.

We use two datasets for experimentation: the IMDB movie review dataset for **binary sentiment analysis** and the SemEval Twitter dataset for **multi-class sentiment classification**. Each sentence is tokenized using the **spaCy** tokenizer, and we construct the vocabulary by filtering words with a minimum frequency threshold of 5. All rare or unseen words during testing are mapped to a special UNK token, and variable-length inputs are standardized using **padding** and **truncation** based on the average sentence length in the dataset. Our input representations are in the form of **one-hot vectors**, and we design fixed input shapes to ensure compatibility with both FFNN and RNN architectures. The FFNN consists of two hidden layers (with 256 and 128 units), while the LSTM model uses a 256-dimensional hidden state for sentence representation. For optimization, we utilize different loss functions for binary and multi-class setups (**Binary Cross-Entropy** and **Categorical Cross-Entropy** respectively), and evaluate model performance using **Accuracy**, **Precision**, **Recall**, and **F1-Score**, both overall and per class. In this study, we also explore the impact of **architectural choices**, **model depth**, and **input representations** on classification performance. Through our implementation, evaluation, and comparative analysis, we aim to highlight the strengths and weaknesses of **feed-forward** and **recurrent** models in sentiment understanding across diverse textual domains.

2 Dataset Preprocessing

The preprocessing of both the IMDB and Twitter datasets follows tailored approaches based on the structure and cleanliness of the source data.

2.1 IMDB Dataset

The IMDB dataset, which contains movie reviews already organized into **positive** and **negative** folders (**train/pos**, **train/neg**, etc.), requires minimal preprocessing. A portion of the dataset (e.g., 20%) is randomly sampled to reduce computational load. The reviews are labeled as 1 for positive and 0 for negative. Optionally, the training data is split into **90% training** and **10% development** sets, and then shuffled for better generalization during model training. The number of examples in train, dev, and test sets is printed for verification. **IMDB Highlights:**

- Clean, labeled data
- Sampling, optional dev split, and shuffling

2.2 Twitter Dataset

The **Twitter** dataset requires **extensive preprocessing** due to its raw and noisy format. It is loaded from a **CSV** file that may lack headers, with columns auto-assigned to text and sentiment. The pipeline removes entries with **missing values** and maps sentiments to numerical labels (**Positive** → 2, **Neutral** → 1, **Negative** → 0), discarding irrelevant labels. The text data undergoes multiple cleaning steps including **lowercasing**, replacing **URLs** with `[URL]`, standardizing mentions to `@user`, **removing punctuation**, and **normalizing whitespace**. Tweets with fewer than 5 characters and duplicate (**text**, **label**) pairs are also removed. Finally, the dataset is sampled based on the defined **sample_size**, and label types are converted to integers. **Twitter Highlights:**

- Extensive cleaning: **URLs**, **mentions**, **punctuation**, **duplicates**
- Label mapping and class filtering
- Short and irrelevant tweets removed

Additionally, **GloVe** embeddings are loaded for text representation, where **unknown tokens** are initialized randomly and **padding tokens** with zeros. The **spaCy** model `en_core_web_sm` is also ensured to be installed for advanced NLP tasks like **tokenization** or **POS tagging**.

2.3 Twitter Dataset Preprocessing Summary

The original dataset consisted of **74,682 entries** and **4 columns**, with the last column containing the tweet text. The data types indicated a mix of integers and strings, with **686 rows** having missing values in the text or sentiment columns—these were removed. Since the CSV lacked headers, appropriate column names were assigned based on structure. Sentiment labels were mapped as follows: **‘Positive’** → 2, **‘Neutral’** → 1, **‘Negative’** → 0. An additional **12,875 rows** with invalid sentiment labels (like **‘Irrelevant’**) were discarded. **Text Cleaning Steps:**

- URLs replaced with `[URL]`
- Mentions normalized to `@user`
- Lowercasing, punctuation removal, and whitespace normalization

This led to an average **text length reduction of 6.71%**.

Additional Cleanup:

- **4,661 duplicate rows** removed (7.63% of data)
- **321 very short tweets** (e.g., **‘was’**, **‘all’**) excluded

The dataset was then sampled with a **20% fraction**, yielding a final size of **11,228 examples**. Labels were successfully converted to integers.

Class Distribution:

- Negative (0): 4,222 → **37.6%**
- Neutral (1): 3,321 → **29.58%**
- Positive (2): 3,685 → **32.82%**

The **class imbalance ratio** was modest at **1.27** (max/min class count).

2.4 Final Stats

- **Average text length:** 106.69 characters
- **Max text length:** 636 characters
- **Total words:** 217,897
- **Unique words:** 18,220
- **Vocabulary richness (unique/total):** 0.08
- **Number of sentiment classes used:** 3

3 Dataset Creation

This report outlines the **text preprocessing**, **vocabulary construction**, and **dataset creation pipeline** used for sentiment analysis tasks, utilizing **GloVe embeddings** and **PyTorch**. The pipeline ensures clean, normalized text input, robust handling of label variations, and efficient preparation of data for deep learning model training.

3.1 Text Tokenization and Preprocessing

The **tokenization** process begins by converting the input text to lowercase to standardize the case. Following this, URLs and **Twitter mentions** are replaced with special tokens (`URL`, `MENTION`) to maintain the structural integrity of the text while abstracting specific user or link information. **Hashtags** are simplified to just the word (e.g., `#happy` becomes `happy`), and emojis and non-word characters are either removed or replaced. Extra whitespace is normalized to ensure consistent spacing. Tokenization is carried out using **spaCy**, a robust natural language processing library. The tokens are further filtered to remove stopwords (common words with little semantic value) and punctuation marks. Additionally, tokens that are shorter than two characters are discarded as they are unlikely to contribute meaningful context for sentiment analysis.

3.2 Vocabulary Construction

The `build_vocabulary()` function is responsible for constructing the vocabulary from the training data. It performs the following steps:

- **Token Counting:** Each token's frequency is counted across the dataset.
- **Frequency Thresholding:** Only tokens that appear at least `min_freq` times (default set to 5) are retained. This helps eliminate noise from infrequent words that are unlikely to generalize well across unseen data.
- **Special Tokens:** Two special tokens are included: `'PAD'` (index 0) for padding sequences to a uniform length, and `'UNK'` (index 1) for words that do not appear in the vocabulary.

The vocabulary serves as the foundation for converting text into numerical representations, where each word is mapped to an index.

3.3 Average Sentence Length Calculation

The `get_avg_length()` function calculates the average number of tokens per sentence in the dataset. This metric is important for determining the appropriate **max_length** for padding or truncating sequences during model training. It ensures that all text inputs are of uniform length, which is required for batch processing in neural networks.

3.4 Text to Index Conversion

The `text_to_indices()` function performs the conversion of text into a sequence of indices. It processes the tokenized text by mapping each token to its corresponding index in the vocabulary. If a token is not present in the vocabulary, it is assigned the index for the `'UNK'` token. Additionally, the function pads the sequences to a specified **max_length**, ensuring that all input sequences are of equal length, which is crucial for batching in deep learning models.

3.5 SentimentDataset Class

The `SentimentDataset` class extends **PyTorch's Dataset** and is used to convert text-label pairs into model-ready input tensors. It takes as input a dataset (list of tuples), the vocabulary, a precomputed GloVe embedding matrix, and classification settings (such as **max_length** and number of classes). The class provides the following functionalities:

- **Label Mapping:** It supports the mapping of string labels (e.g., `"Positive"`, `"Negative"`, `"Neutral"`) to integer labels. This allows flexibility in handling various label formats. If the labels are provided in numeric format or string format, they are converted to integers as needed.
- **Error Handling:** If a label cannot be mapped or converted to a number, it defaults to 0 to prevent runtime errors. This ensures the dataset remains robust even with inconsistent label formats.
- **Tensor Conversion:** Labels are converted to appropriate PyTorch tensor types: `float32` for binary classification and `long` for multiclass classification tasks. The input sequences are also converted to tensors for compatibility with PyTorch models.

3.6 Summary of Key Features

- **Robust Tokenization:** Utilizes **spaCy** for advanced tokenization and noise handling, including URL replacement, mention normalization, and punctuation removal.
- **Efficient Vocabulary Construction:** The vocabulary is built based on word frequencies, with special handling for rare words and the inclusion of necessary special tokens (`PAD`, `UNK`).
- **Label Flexibility:** The pipeline supports various label formats, ensuring robust handling of inconsistent or non-standardized labels.

- **Indexing and Padding:** Text is converted into indexed sequences and padded to ensure uniform input sizes for neural networks.
- **Dataset Integration:** The custom `SentimentDataset` class integrates seamlessly with `PyTorch DataLoader` for efficient batch processing and model training.

This preprocessing pipeline prepares the data efficiently and ensures that the model receives clean, structured input, ready for training on sentiment classification tasks, whether for binary or multiclass classification.

4 Feed Forward Network

This report outlines the development of a **Feed-Forward Neural Network (FFNN)** model and its training pipeline for sentiment analysis, utilizing pre-trained **GloVe embeddings** and the **PyTorch** framework. The model is designed to perform both binary and multiclass sentiment classification. The architecture includes an embedding layer to leverage word embeddings, followed by two fully connected layers with batch normalization, ReLU activations, and dropout to regularize the network. The network's outputs are processed using a **sigmoid** activation function for binary classification and **softmax** for multiclass classification. **Key Components**

4.1 FeedForwardNN Class:

The core model consists of the following components:

- **Embedding Layer:** The model initializes an embedding layer using pre-trained **GloVe embeddings** (if provided), allowing the model to leverage external word vector representations, which are pre-loaded and fine-tuned during training.
- **Fully Connected Layers:** The network contains two hidden layers, each with ReLU activations, batch normalization, and dropout for regularization. These layers are designed to learn complex patterns in the data while mitigating overfitting risks.
- **Output Layer:** The model includes an output layer that produces predictions either via **sigmoid** (for binary classification) or **softmax** (for multiclass classification), depending on the nature of the sentiment analysis task.

4.2 Model Training:

- **Training Process:** The model is trained using the `train_model` function. This function trains the model for multiple epochs, calculating loss with binary cross-entropy (**BCELoss**) for binary classification or cross-entropy loss for multiclass classification.
- **Early Stopping:** The training process includes early stopping, which halts training if the model's performance on the validation set fails to improve after a set number of epochs (patience). This helps prevent overfitting and ensures efficient training.

4.3 Evaluation:

- **Evaluation Metrics:** The `evaluate_model` function computes key performance metrics such as accuracy and loss on the validation or test datasets.
- **Detailed Reporting:** The `evaluate_and_report` function provides detailed metrics, including precision, recall, and F1 score for binary classification tasks. For multiclass classification, a classification report is generated, offering more detailed insights into model performance across different classes.

4.4 Model Comparison and Visualization:

- **Model Comparison:** The FFNN model's performance is compared with an **LSTM model**. The comparison is facilitated by visualizing both models' training and validation losses, as well as their accuracy curves over the training epochs.
- **Bar Chart Comparison:** A final bar chart presents the accuracy and loss for both the FFNN and LSTM models at the last epoch, providing a clear comparison of the models' effectiveness on the IMDB test set.

5 LSTM

This report outlines the process of performing sentiment analysis on a Twitter dataset using a **Long Short-Term Memory (LSTM)**-based neural network. The model incorporates 5-fold cross-validation

to assess its generalization performance and improve reliability. The script is designed to handle data preprocessing, model training, evaluation, and result visualization in a streamlined workflow.

5.1 Key Components

5.1.1 Model Architecture (LSTMNN Class)

The model architecture consists of several key layers designed to handle sequential text data and enhance learning performance:

- **Embedding Layer:** The model begins with an embedding layer that either learns word embeddings from scratch or uses pre-trained embeddings (e.g., **GloVe**) if provided. The embedding weights are trainable, enabling the model to learn and adapt word representations during training.
- **LSTM Layer:** After embedding, the sequence data passes through an LSTM layer. LSTM units are well-suited for sequential data, as they preserve long-term dependencies and allow the model to capture the context and relationships within text sequences.
- **Batch Normalization:** To improve training speed and model performance, a batch normalization layer is applied to the output of the LSTM layer. This helps stabilize training by normalizing the activations, reducing the risk of overfitting.
- **Dropout Layer:** A dropout layer is included after batch normalization to prevent overfitting. It randomly drops a fraction of the neurons during training, ensuring the model does not rely too heavily on any single feature.
- **Output Layer:** The output layer is tailored for the type of classification:
 - **Binary Classification:** For binary sentiment classification, the output layer consists of a single neuron with a **sigmoid** activation function, producing a probability score.
 - **Multi-Class Classification:** For multi-class sentiment classification, the output layer includes neurons equal to the number of classes, with **softmax** activation for producing a probability distribution over all possible classes.

5.1.2 Cross-Validation Setup (perform_cross_validation Function)

To evaluate the model's performance reliably, 5-fold cross-validation is employed. The dataset is split into five subsets, and the model is trained and validated on each fold to assess its generalization ability.

- **Data Splitting:** In each fold, the model is trained on 80% of the data (training set) and evaluated on the remaining 20% (validation set). This ensures that the model is assessed on different subsets of the data, providing a robust estimate of its performance.
- **Vocabulary and Embedding Matrix:** The training data is used to create a vocabulary, and **GloVe** word embeddings are loaded to initialize the model's embedding layer. These pre-trained embeddings offer a richer representation of words, improving learning efficiency.
- **Dataset and Dataloader:** The custom dataset class **SentimentDataset** is used to handle the data for both training and validation. Dataloaders are employed to manage batch processing during training, facilitating efficient data loading.
- **Model Training:** The model is trained for 10 epochs using the Adam optimizer and **CrossEntropyLoss** (for multi-class classification) or **BCELoss** (for binary classification). Early stopping is applied, halting training if the validation loss does not improve for 3 consecutive epochs, preventing overfitting.

5.1.3 Model Evaluation

Once the model is trained, it is evaluated on the validation set. The evaluation metrics used are as follows:

- **Accuracy:** Measures the overall correctness of the model's predictions.
- **Precision, Recall, F1 Score:** These metrics are computed to assess the model's ability to classify each class correctly, particularly when dealing with imbalanced classes.

For binary classification, the model's output is thresholded at 0.5 to determine class predictions. For multi-class classification, **argmax** is used to select the class with the highest predicted probability.

5.1.4 Visualization of Training Curves

Training progress is monitored by plotting the following metrics:

- **Training and Validation Loss:** These curves help visualize how well the model is learning and whether it is overfitting or underfitting.
- **Validation Accuracy:** The accuracy curve shows the model's ability to generalize to the validation set.

The training curves for each fold are plotted, and average curves are displayed for both loss and accuracy, providing a comprehensive view of the model's performance across all folds.

5.1.5 Final Results

After completing the 5-fold cross-validation process, the average performance metrics (accuracy, precision, recall, and F1 score) are calculated and reported. These metrics provide an overall summary of the model's ability to perform sentiment analysis on the Twitter dataset.

- **Training Curves:** The training curves (loss and accuracy) are saved as a PNG file for further analysis of the training process.

5.2 Conclusion

This script provides a robust framework for training and evaluating an **LSTM-based neural network** for sentiment analysis on the Twitter dataset. By utilizing 5-fold cross-validation, the model's generalization ability is carefully assessed, and overfitting is mitigated through techniques like early stopping, batch normalization, and dropout. Detailed evaluation metrics and visualizations offer valuable insights into model performance and can guide further improvements or model tuning for better results.

6 Training and Evaluations : LSTM vs. FFNN

section Training and Evaluations: LSTM vs. FFNN

6.1 LSTM Model Evaluation (5-Fold Cross-Validation)

The model was evaluated using 5-fold cross-validation on a sentiment analysis task. Below are the results from each fold and the overall performance:

- **Fold 1:** Accuracy = 0.6883, Precision = 0.6880, Recall = 0.6803, F1 Score = 0.6817
- **Fold 2:** Accuracy = 0.6598, Precision = 0.6562, Recall = 0.6528, F1 Score = 0.6527
- **Fold 3:** Accuracy = 0.6541, Precision = 0.6631, Recall = 0.6575, F1 Score = 0.6518
- **Fold 4:** Accuracy = 0.6784, Precision = 0.6841, Recall = 0.6706, F1 Score = 0.6721
- **Fold 5:** Accuracy = 0.6837, Precision = 0.6826, Recall = 0.6795, F1 Score = 0.6787

The average accuracy across all folds was 0.6729, with an average precision of 0.6748, average recall of 0.6681, and average F1 score of 0.6674. The LSTM model showed relatively stable performance across the different folds, with accuracy ranging from 65% to 69% and an F1 score of 0.6674. This indicates moderate overall performance, but there is room for improvement, particularly in terms of recall.

6.2 Comparison: Feed-Forward Neural Network (FFNN) vs. LSTM

6.2.1 Feed-Forward Neural Network (FFNN) Results

The FFNN model was trained for 10 epochs. The training loss decreased from 0.6687 at epoch 1 to 0.0000 by epochs 9-10. The development accuracy improved from 0.6600 at epoch 1 to 0.6920 by epoch 10. On the test set, the FFNN achieved an accuracy of 0.6746, precision of 0.6722, recall of 0.6816, and an F1 score of 0.6769. The FFNN showed decent performance, but struggled with precision, as evidenced by the fact that recall (0.6816) was slightly higher than precision (0.6722).

6.2.2 LSTM Results

The LSTM model was also trained for 10 epochs, with the training loss decreasing from 0.6896 at epoch 1 to 0.0280 at epoch 10. The development accuracy improved from 0.5780 at epoch 1 to 0.7300 by epoch 10, indicating that the LSTM model is better at learning sequential patterns in the data. On the test set, the LSTM achieved an accuracy of 0.7038, precision of 0.7703, recall of 0.6628, and an F1 score of 0.7125. The LSTM outperformed the FFNN in terms of accuracy and F1 score, but still exhibited a tradeoff between precision and recall, with precision being higher than recall.

6.3 Conclusion & Insights

1. **LSTM Outperforms FFNN:** The LSTM model consistently outperforms the FFNN, demonstrating better performance in both accuracy and F1 score. The LSTM's ability to capture sequen-

tial dependencies in data is key to its superior performance in sentiment analysis tasks.

2. **Precision-Recall Tradeoff:** Both models show a tradeoff between precision and recall, with the LSTM being more precise but struggling with recall. This indicates a potential class imbalance or difficulty in identifying all relevant positive instances.
3. **Potential Improvements:** Strategies such as hyperparameter tuning, class weighting, and data augmentation could help improve the models, particularly in terms of recall.

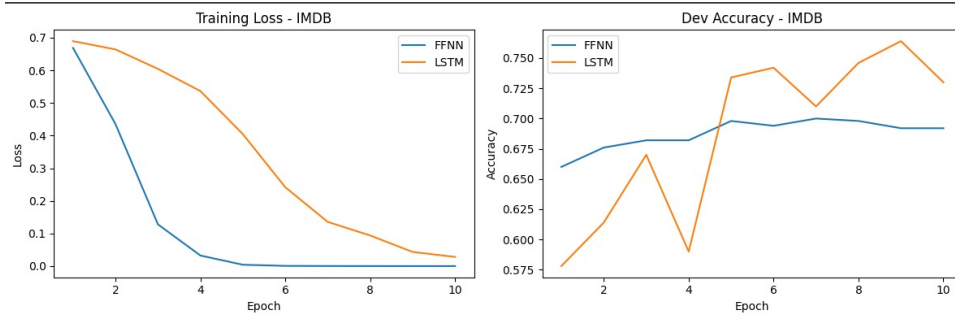


Figure 1: Performance Comparison: LSTM vs. FFNN

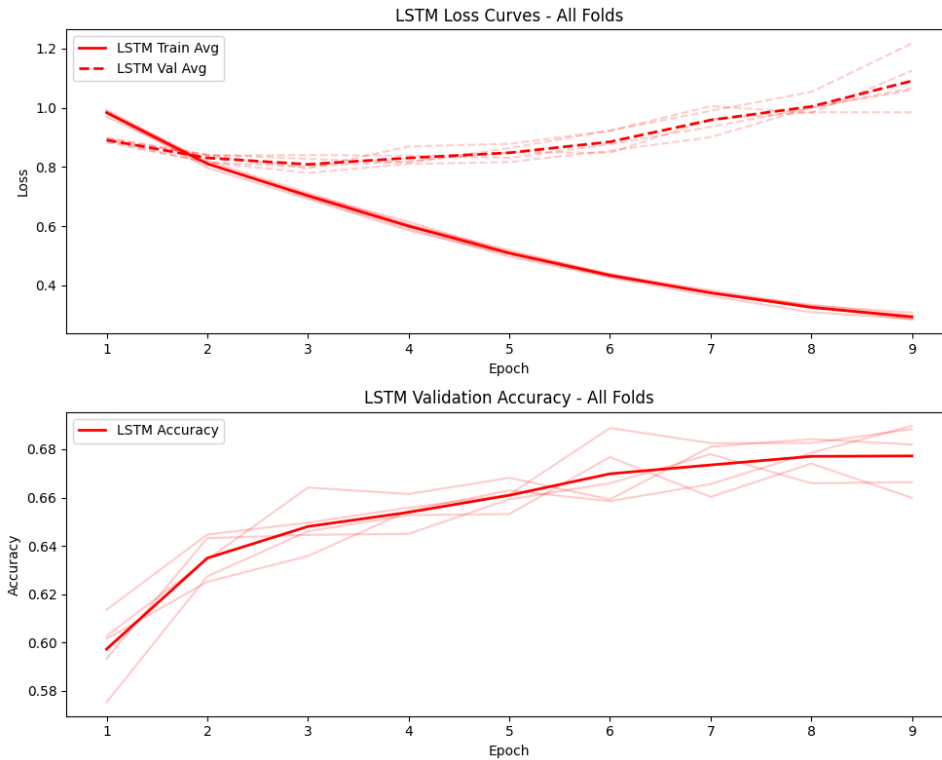


Figure 2: Learning Curves: Loss and Accuracy During Training

7 Here is the Kaggle Code link: Colab Link